



SIMICO Warehouse Scan

REST API Integration Guide — for WMS / ERP implementation

Document version 1.0 · 23 August 2026

Audience: backend / WMS-ERP developers implementing the endpoints this Android app calls.

Companion apps: SIMICO Warehouse Scan (Android, Capacitor + React) — github.com/simicosrl/mobile

Transport	HTTPS, JSON request/response bodies
Authentication	Bearer token — Authorization: Bearer <api key>
Base URL	Configured per install, in the app's API screen (no fixed host)
Endpoints to implement	4 — 2 the app calls to send data, 2 it calls to receive data
Timeout	App aborts and treats as failed after 15 seconds
Verified	End-to-end against a reference mock server for this document — see §10

1. Overview

SIMICO Warehouse Scan is an Android app operators use on the warehouse floor (Zebra handhelds, phones, tablets) to record every parcel entering and leaving the building: scan tracking IDs, flag damage with a photo, capture the driver's signature, and print an A4 handover document. This guide documents the REST API surface it can talk to, so your WMS/ERP can receive that data automatically and, optionally, feed the app an expected-parcel manifest.

There is no bundled backend. The app is fully usable standalone (all data lives on-device); the API is an optional integration layer an operator turns on from the app's **API** tab by entering your endpoint's base URL and an API key. Nothing is sent anywhere until that's configured.

Nothing to install on your side beyond the four endpoints below. The app never assumes a specific backend framework — implement them however suits your WMS.

1.1 Data flow

Outbound (app → WMS): a confirmed inbound/outbound session, and optionally a damage record the moment it's captured.

Inbound (WMS → app): today's expected tracking-ID manifest, and the WMS's own archived copy of a handover document's PDF.

2. Authentication & configuration

Every request the app makes carries these headers:

```
Authorization: Bearer <api key>
Content-Type: application/json
```

The base URL and API key are entered by the operator in the app (Settings → API tab) and stored on-device only — there is no shared secret baked into the app binary. Two independent switches control direction:

- **Send sessions automatically** — POST every confirmed document (and each damage record) as it happens.
- **Receive expected manifest** — GET today's manifest on demand (manual “Pull manifest now” button); pulled codes are compared against every scan.

2.1 Timeouts & retries

Every request aborts after **15 seconds** and is treated as a failure. There is no automatic retry loop — a failed outbound document sits in a **failed** queue on the app's API tab and History, with a **Retry** button per document (re-sends the identical payload) and a **Sync now** button that retries every non-**ok** document at once.

2.2 Response parsing — please read this if nothing else

The app parses any response body that is valid JSON as JSON, regardless of the **Content-Type** header you send. You do not need to worry about getting that header exactly right for the app's sake — but please set it correctly anyway for every other HTTP client. A response that fails to POST (body-parsing aside) is judged purely by its HTTP status code: anything outside 200–299 is a failure.

3. Send a confirmed session

POST

/warehouse/sessions

Called once, immediately after the operator confirms a handover (signs and taps **Confirm handover**) — if **Send sessions automatically** is on. This is the single most important endpoint: the full record of one inbound or outbound session, its parcel list, any damage, and the driver's signature.

Request body

Field	Type	Req.	Description
document	string	yes	Document number, e.g. SM-IN-000241 / SM-OUT-000241. Generated client-side, sequential per direction. Treat as the durable external key for this session.
direction	string	yes	"inbound" or "outbound".
carrier	string	yes	Carrier code selected for the session, e.g. DHL, UPS, GLS, DPD, TNT, FEDEX, AMAZON, OTHER.
driver.name	string null	no	Driver's printed name, captured on the signature screen.
driver.plate	string null	no	Vehicle plate, free text (uppercased in the app's UI).
operator	string	yes	The signed-in operator's badge ID, e.g. BADGE-4471.
closed_at	string	yes	ISO-8601 timestamp of confirmation, device-local time.
parcels	array	yes	One entry per scanned parcel, in scan order — see 3.1.
signature	string null	no	Driver's signature as a base64 PNG data URL (data:image/png;base64,...).

3.1 Parcel object (inside `parcels[]`)

Field	Type	Req.	Description
tracking	string	yes	Tracking ID as scanned, trimmed and upper-cased.
boxes	integer	yes	Box count for this line (operator can bump it with +/-, or via "+1 box" on a duplicate scan).
expected	boolean null	yes	true/false if a manifest was pulled and this code was checked against it; null if manifest-checking was off.
damage	object null	yes	null if undamaged, else {"type": string, "photo": string null} — see §4 for how the photo itself arrives.

Example request

```
POST /warehouse/sessions HTTP/1.1
Authorization: Bearer sk_live_9f2c41ab7d0e5583
Content-Type: application/json
{
  "document": "SM-IN-000241",
  "direction": "inbound",
  "carrier": "DHL",
  "driver": { "name": "Luca Ferri", "plate": "GA 481 KP" },
```

```

"operator": "BADGE-4471",
"closed_at": "2026-08-23T09:41:00+02:00",
"parcels": [
  { "tracking": "JJD014600011234567890", "boxes": 1,
    "expected": true, "damage": null },
  { "tracking": "TBA303120456789", "boxes": 2,
    "expected": false,
    "damage": { "type": "Seal broken",
                "photo": "IMG_0442.jpg" } }
],
"signature": "..."
}

```

Expected response

Code	Meaning to the app
200–299	Success. The app marks the document sent .
anything else	Failure. The document is queued as failed with a Retry button; the same payload is re-sent verbatim on retry.
(timeout)	No response within 15s counts as failure, same as above.

A response body is not required; if you do return JSON, its shape isn't currently used by the app, so return whatever's natural for your system (e.g. `{"id": "..."}).`

4. Push a damage record

POST

/warehouse/parcels/{tracking}/damage

Called the moment the operator taps **Save damage** on a parcel — independently of, and before, the session itself is confirmed and sent. {tracking} is the same tracking ID that also appears in that parcel's line inside the eventual session payload (§3.1) — the same damage will be visible in both places.

Request body

Field	Type	Req.	Description
tracking	string	yes	Repeats the path parameter, for convenience.
type	string	yes	Comma-joined damage types plus an optional free-text note, e.g. "Box damaged, Seal broken – corner crushed". Base types: Box damaged, Seal broken, Parcel opened.
photo	string	yes	The captured photo as a base64 data URL (data:image/jpeg;base64, ... — a real photo is mandatory in the app before this fires).
recorded_at	string	yes	ISO-8601 timestamp of capture.

Expected response

Code	Meaning to the app
200–299	Success — no further app-side action.
anything else / timeout	The app surfaces a one-line toast naming the tracking ID; it does not block or retry automatically — the damage still goes out again as part of §3 when the session is confirmed.

This endpoint is a nice-to-have for near-real-time damage alerts to your office; it is not the only way damage data reaches you. Every damage record also always travels inside the session payload (§3.1), so implement §3 first if you can only build one endpoint initially.

5. Provide today's expected manifest

GET

/warehouse/manifest?date=today

Called when the operator taps **Pull manifest now** on the API tab (manual — there is no background polling). Returns every tracking ID your WMS expects to move today, across all carriers. The app doesn't filter by direction or carrier on its side — send the full day's list.

Response body — either shape is accepted

```
// Shape A – a bare array
["JJD014600011234567890", "1Z999AA10123456784", "..."]
// Shape B – wrapped in an object
{ "tracking_ids": ["JJD014600011234567890", "1Z999AA10123456784"] }
```

Once pulled, every scan for the rest of that session is checked against this list: a match sets that parcel's expected field to true and shows an **Expected** chip in the app; anything else shows **Not on manifest** but is still accepted normally — an unknown tracking ID is never blocked.

Expected response

Code	Meaning to the app
200 + array /object above	Success — manifest count and “last pull” time update in the app.
anything else / timeout / unparseable body	Pull fails; the app shows the previous manifest (or none) and a toast with the error.

6. Serve back the archived handover PDF

GET

/warehouse/sessions/{document}/pdf

Called when the operator taps **Print A4 document**, **View document**, **Download PDF** or **Reprint document** for a session whose sync status is already **ok** (i.e. §3 succeeded for it). {document} is the same document number from §3.

This lets your WMS be the source of truth for the PDF — useful if it re-stamps an official document number, letterhead, or legal footer that differs from the app's own locally-generated version. **The app always has a working fallback**: if this call fails, isn't implemented, or the base URL isn't configured, it silently falls back to generating the same A4 document locally with j sPDF — implementing this endpoint is entirely optional.

Response

Code	Meaning to the app
200 + PDF bytes	Content-Type: application/pdf (or anything — the app reads the raw bytes as a blob regardless of header, base64-encodes them, and hands them to Android's native share sheet).
anything else / timeout	The app falls back to generating its own PDF locally — the operator sees no error.

7. Reference: enumerations

7.1 Direction

"inbound" — receiving from a courier. "outbound" — handing over for pickup.

7.2 Carriers

DHL, UPS, GLS, DPD, TNT, FEDEX, AMAZON, OTHER — extend this list in the app's source (src/lib/carriers.js) if you need more; the API accepts whatever string the app sends without a fixed enum on your side.

7.3 Damage types

Box damaged, Seal broken, Parcel opened — one or more, joined with a comma, plus an optional free-text note appended after an em dash.

7.4 Document numbering

Format SM-IN-NNNNNN / SM-OUT-NNNNNN, zero-padded to 6 digits, generated and incremented on-device per direction. This is **not** coordinated with your WMS — treat the document field as an opaque external key, and do not expect it to align with any sequence your own system maintains.

8. Error handling, from the app's point of view

There is no special-cased error format to implement. The app only distinguishes success (HTTP 2xx) from failure (anything else, a network error, or no response within 15 seconds) — an error response body is never parsed for a message to show the operator. If you want operators to see something actionable, keep the

failure visible on your side (e.g. an alert to your ops team) rather than relying on the app to surface your error text.

Failed outbound sends are never silently dropped: they stay queued on the app's API tab with document number, carrier, and parcel/box counts visible, each with its own Retry button, until they succeed or the operator gives up.

9. Security notes

- Use HTTPS. The app does not restrict or warn on plain HTTP, but treat that as a development-only convenience.
- The API key is a single static Bearer token per install, entered by the operator and stored only on-device — there is no token refresh/expiry flow. Rotate it by having the operator re-enter a new one in the app.
- Signatures and damage photos travel as base64 inside JSON bodies, not as separate multipart uploads or file references — size your request-body limits accordingly (a single signature PNG is typically a few KB; a phone photo can be 1–5 MB).

10. How this was verified

Every endpoint in this document was exercised end-to-end against a disposable reference HTTP server before this guide was written — not just read from source. That pass found and fixed two real gaps versus what was originally documented:

- **§4 (damage push) and §6 (archived PDF fetch) were defined in the app's client code but never actually called.** Both are now wired in: damage pushes on save when auto-send is on, and printing/downloading a synced document tries the ERP's archived copy first.
- **Response parsing was silently gated on a correct Content-Type header**, so a JSON body sent without one (a common real-world misconfiguration) was treated as unparseable and the whole call reported as failed. Fixed to parse any body that is valid JSON, header or not (§2.2).

After those fixes, a full pass — pull manifest, scan an expected and an unexpected code, flag damage with a real captured photo, confirm and auto-send the session, then fetch the archived PDF back — completed correctly against the reference server, including confirming the returned PDF bytes arrive byte-for-byte intact (no base64/binary corruption).

Appendix: full endpoint quick-reference

Method	Path	Direction	Purpose
POST	/warehouse/sessions	App → WMS	One confirmed session
POST	/warehouse/parcels/{tracking}/damage	App → WMS	One damage record, on capture
GET	/warehouse/manifest?date=today	WMS → App	Today's expected tracking IDs
GET	/warehouse/sessions/{document}/pdf	WMS → App	Archived PDF (optional; app has a local fallback)

Questions about this integration: contact the SIMICO Warehouse Scan app maintainer via the [simicosrl/mobile](#) repository on GitHub.